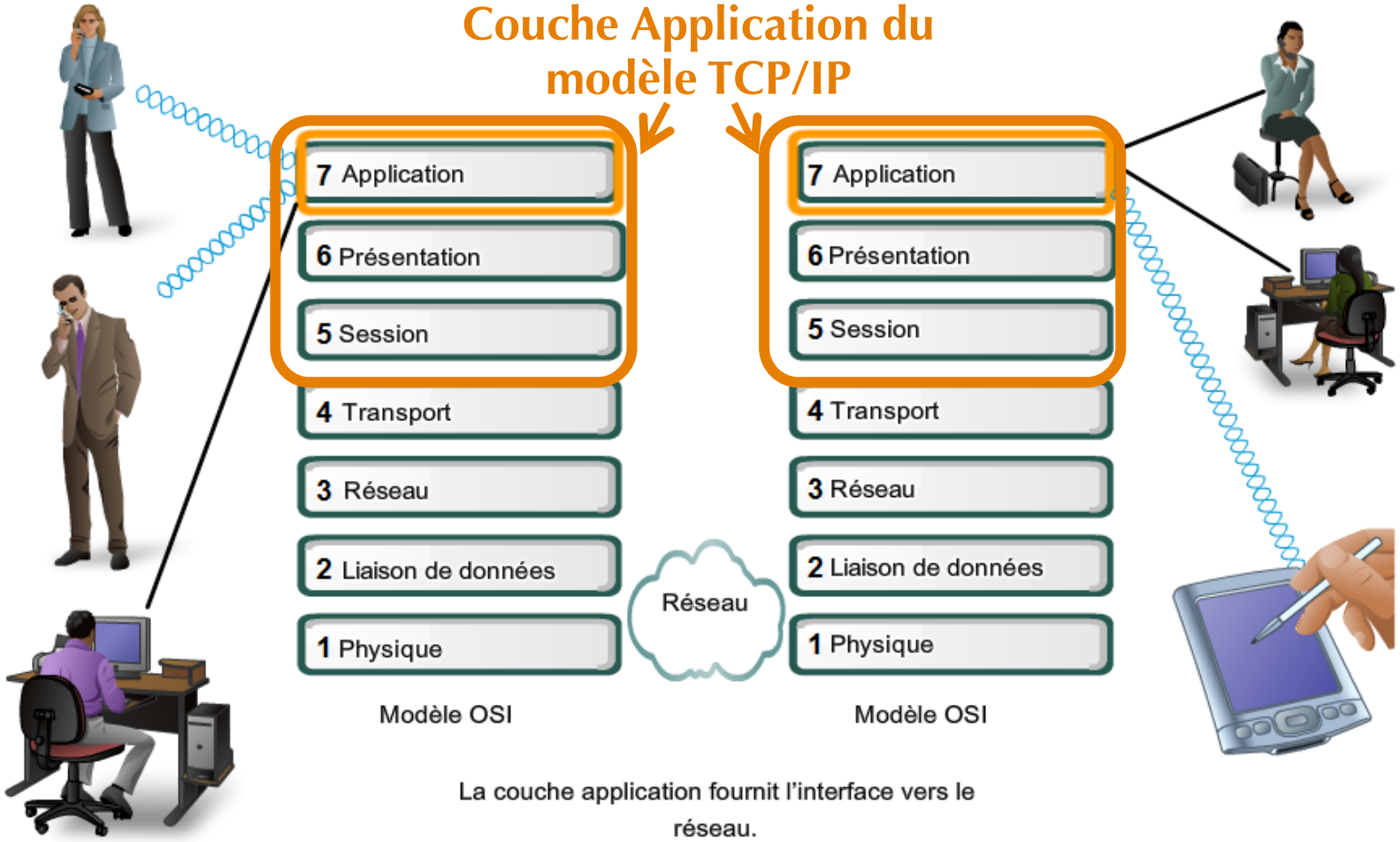
La couche application

La couche application

- La **couche Application du modèle TCP/IP** englobe les **couches 5, 6 et 7 du modèle OSI** :

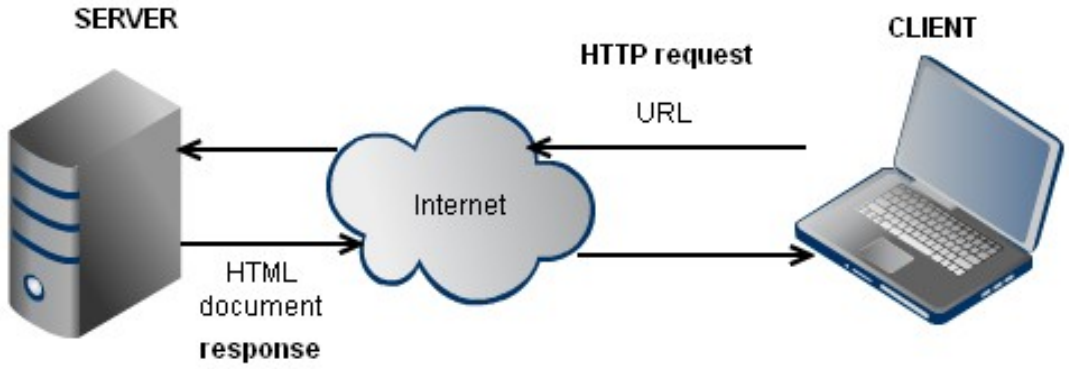


La couche application fournit l'interface vers le réseau.

Protocole : HTTP

Protocole : HTTP – Le principe

- **HTTP** : **H**yper**T**ext Transfert **P**rotocol
- HTTP est un protocole de communication **entre le navigateur du client** et **les serveurs Web.** Il permet le transfert de fichier (essentiellement au format HTML) localisé grâce à son **URL** entre client et un serveur Web.



- La communication entre le navigateur et le serveur se fait en deux temps :
  - le navigateur effectue une **requête** HTTP ;
  - le serveur traite la requête puis envoie une **réponse** HTTP.

Protocole : HTTP – Le principe

Exemple : L'affichage d'une page web dans un navigateur en utilisant l'URL http://www.cisco.com/index.html.

Étape 1

Le navigateur commence par interpréter les trois parties de l'adresse URL :

- http (le protocole ou schéma)
- www.cisco.com (nom du serveur)
- index.html (nom du fichier demandé)

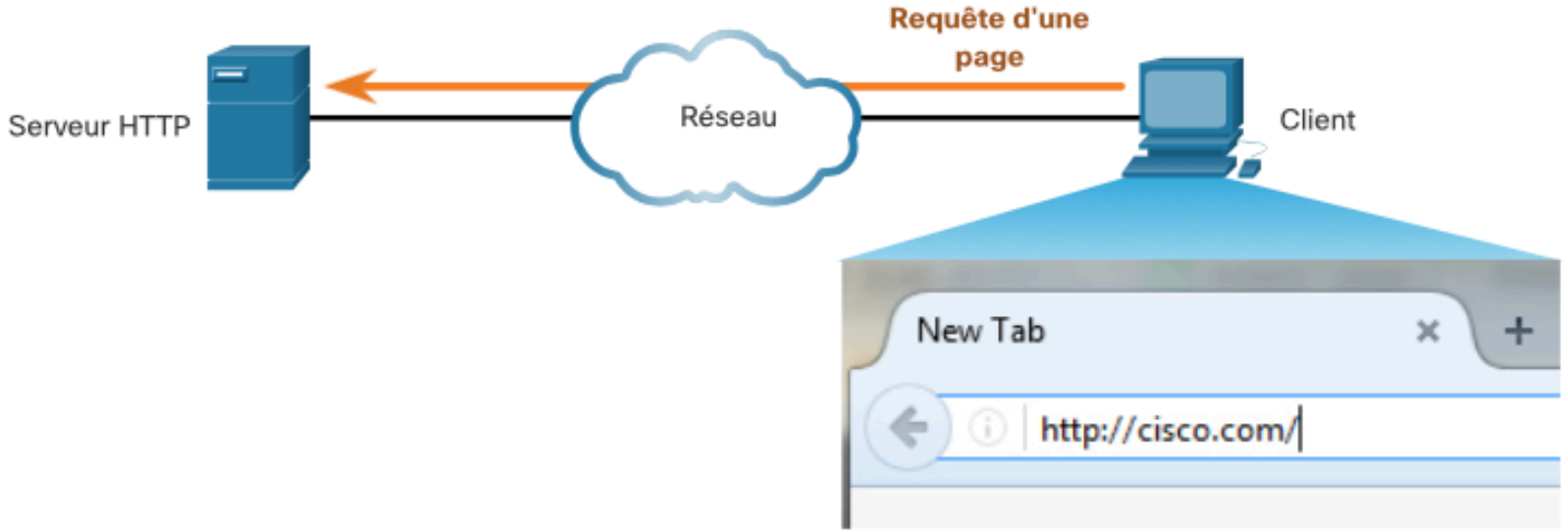


Protocole : HTTP – Le principe

- **Étape 2**

Le navigateur vérifie ensuite avec un serveur de noms pour convertir l'adresse www.cisco.com en une adresse numérique, qu'il utilise pour se connecter au serveur.

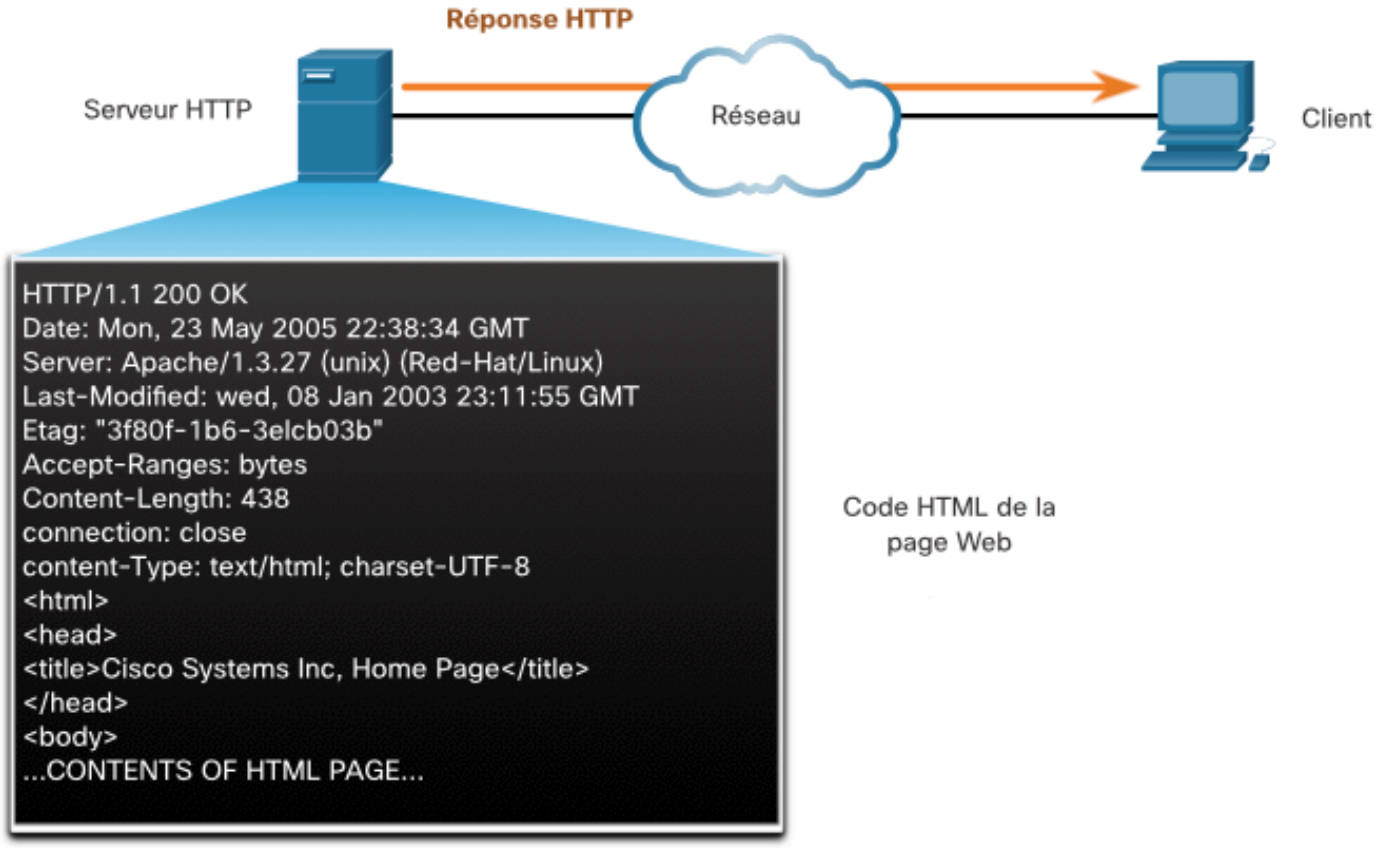
Le client initie une requête HTTP à un serveur en envoyant une requête **GET** au serveur et demande le fichier index.html.



Protocole : HTTP – Le principe

Étape 3

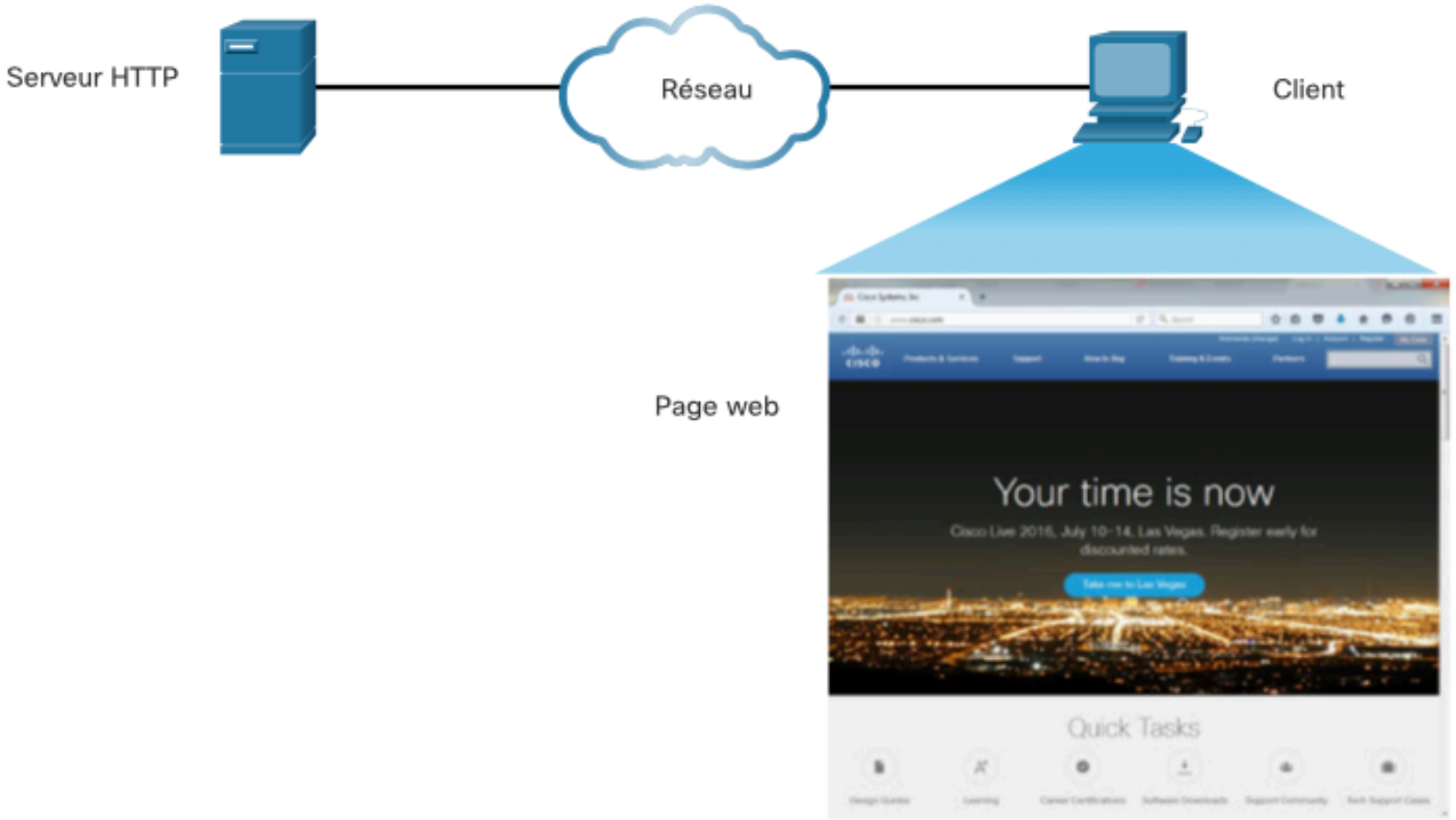
- En réponse à la demande, le serveur envoie le **code HTML** de cette page Web au navigateur.



Protocole : HTTP – Le principe

Étape 4

- Le navigateur déchiffre le code HTML et met en forme la page pour la fenêtre du navigateur



La couche application

Protocole : HTTP – L'URL

- **URL** (**U**niform **R**essource **L**ocator) :

protocol://[user:password@]host[:port][/path][/document]

  - user : nom d'utilisateur utilisé par certains protocoles (FTP...)
  - password : mot de passe pour user
  - host : nom complet d'hôte FQDN (www.education.gouv.fr)
  - port : port sur lequel se connecter, la plupart des protocoles ont un port par défaut (80 pour HTTP...)
  - path : détail sur la façon d'accéder à la ressource (/~rep)
  - document : nom de la ressource (index.html, video.mpeg...)

- Exemple : http://www.cisco.com/index.html.

Protocole : HTTP – Requête HTTP

- Syntaxe d'une requête HTTP :

```
METHODE URL VERSION
EN-TETE : Valeur
EN-TETE : Valeur

CORPS DE LA REQUETE
```

- METHODE URL VERSION : La ligne de requête comprend 3 éléments séparés par un espace:
  - La méthode (**GET**, **POST, PUT**..) ;
  - L'URL ;
  - La version du protocole utilisé par le client (HTTP/1.1 par exemple).
- EN-TETE : Les champs d'en-tête est un ensemble de lignes facultatives permettant de communiquer des informations supplémentaires sur la requête et/ou le client (navigateur, OS...).
- CORPS DE LA REQUETE : Le corps de la requête est un ensemble de lignes optionnel devant être séparé des lignes précédentes par une ligne vide (par exemple pour un envoi de données par la méthode POST lors de l'utilisation de formulaire).

La couche application

Protocole : HTTP – Requête HTTP

- Méthode **GET**

==> Demander des informations au serveur sans modifier quoi que ce soit

```
GET / HTTP/1.1\r\n
Host: www.voila.fr\r\n
User-Agent: Mozilla/5.0 (Windows; U; Windows NT 6.1; fr; rv:1.8.1.6) Gecko/20070725 Firefox/3.6.13\r\n
Accept: text/xml,application/xml,application/xhtml+xml,text/html;q=0.9,text/plain;q=0.8\r\n
Accept-Language: fr,fr-fr;q=0.8,en-us;q=0.5,en;q=0.3\r\n
Accept-Encoding: gzip,deflate\r\n
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7\r\n
Keep-Alive: 300\r\n
Connection: keep-alive\r\n
Cookie: PersoHome=d:4&nVer:1.2; PersoModRecherche=Cd:acc_search_sld1&nVer:1.2; \r\n
\r\n
```

Protocole : HTTP – Requête HTTP

La méthode POST :

==> Envoyer des données (création)

- Tu remplis un formulaire d'inscription sur un site.
- Quand tu cliques sur "Envoyer", ton navigateur envoie une requête **POST** avec tes infos (nom, email…).
- Le serveur crée un nouveau compte.

```
POST /register HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "name": "nom",
  "email": " nom@example.com"
}
```

La méthode PUT

==> Mettre à jour des données

Exemple concret :

- Tu veux modifier ton profil utilisateur.
- L'application envoie une requête **PUT** avec les nouvelles infos.
- Le serveur met à jour ton profil.

```
PUT /users/123 HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "name": "nom prenom",
  "email": " nom.prenom@example.com"
}
```

Protocole : HTTP – Réponse HTTP

- Syntaxe d'une réponse HTTP :

```
VERSION-HTTP CODE INDICATION
EN-TETE : Valeur
EN-TETE : Valeur

CORPS DE LA REPONSE
```

- VERSION-HTTP CODE INDICATION : Comprend 3 éléments:
  - version HTTP ;
  - un code d'état décrivant le résultat (200 pour un acquittement, 400 , 404...) ;
  - Une chaîne de caractère donnant une indication sur le résultat (OK, Bad request, Not Found, Athorization Required...)

- EN-TETE : Les champs d'en-tête ensemble de lignes facultatives permettant de communiquer des informations supplémentaires sur la réponse (type du serveur, ...)

- CORPS DE LA REPONSE : qui contient les données HTTP au format html; php....

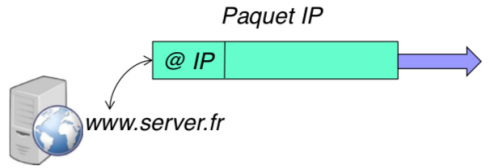
Protocole : HTTP - Réponse HTTP

- Exemple :

```
HTTP/1.1 200 OK\r\n
Date: Thu, 13 Sep 2010 09:59:46 GMT\r\n
Server: Apache/2.0.50 (Unix) mod_ssl/2.0.50 OpenSSL/0.9.7c\r\n
Last-Modified: Mon, 03 Sep 2010 12:52:39 GMT\r\n
Accept-Ranges: bytes\r\n
Content-Length: 5228
Keep-Alive: timeout=15, max=99\r\n
Connection: Keep-Alive\r\n
Content-Type: text/html\r\n
Content-Language: fr\r\n
\r\n
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"\r\n
 "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">\r\n
<html xmlns="http://www.w3.org/1999/xhtml">\r\n
<!-- DW6 -->\r\n
<head>\r\n
...
```

Protocole : DNS

Le service DNS – Le principe



- Pour circuler sur Internet, un paquet IP doit contenir l'adresse IP de destination et non le nom associé**.**
- Pour simplifier l'identification, un service de résolution permettant d'utiliser **des noms symboliques** de machines à la place **des adresses IP** est utilisé sur tous les réseaux TCP/IP.
- Ces **noms symboliques** sont des **noms de domaines.**
- Le système de résolution est appelé **DNS**: **D**omain **N**ame System.
- Sur Internet, la gestion des noms est centralisée sur des **machines spécifiques** (**les serveurs de noms**) à l'aide du service DNS permettant une organisation **hiérarchisée arborescente.**

- **hiérarchisée arborescente ?**

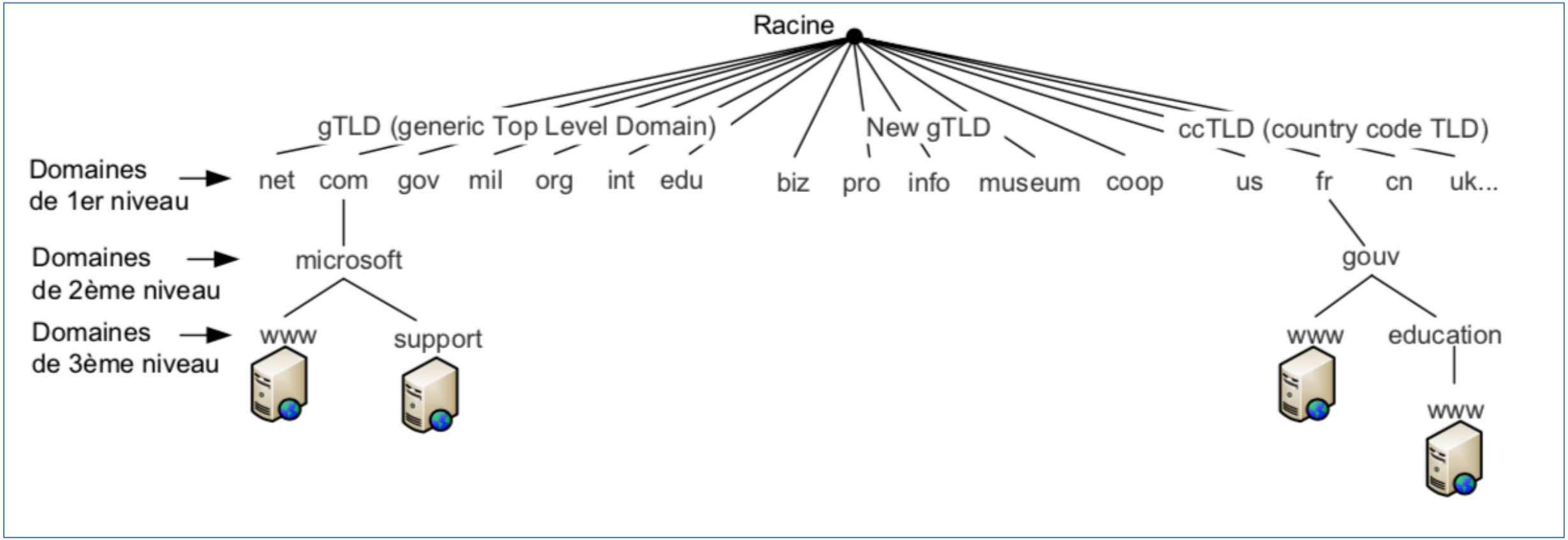
Le service DNS – L'arborescence

- **FQDN** (**F**ully **Q**ualified **D**omain **N**ame) = Un nom complet d'hôte = est constitué des domaines successifs séparés par des points, et il inclut implicitement (ou explicitement) le domaine racine.

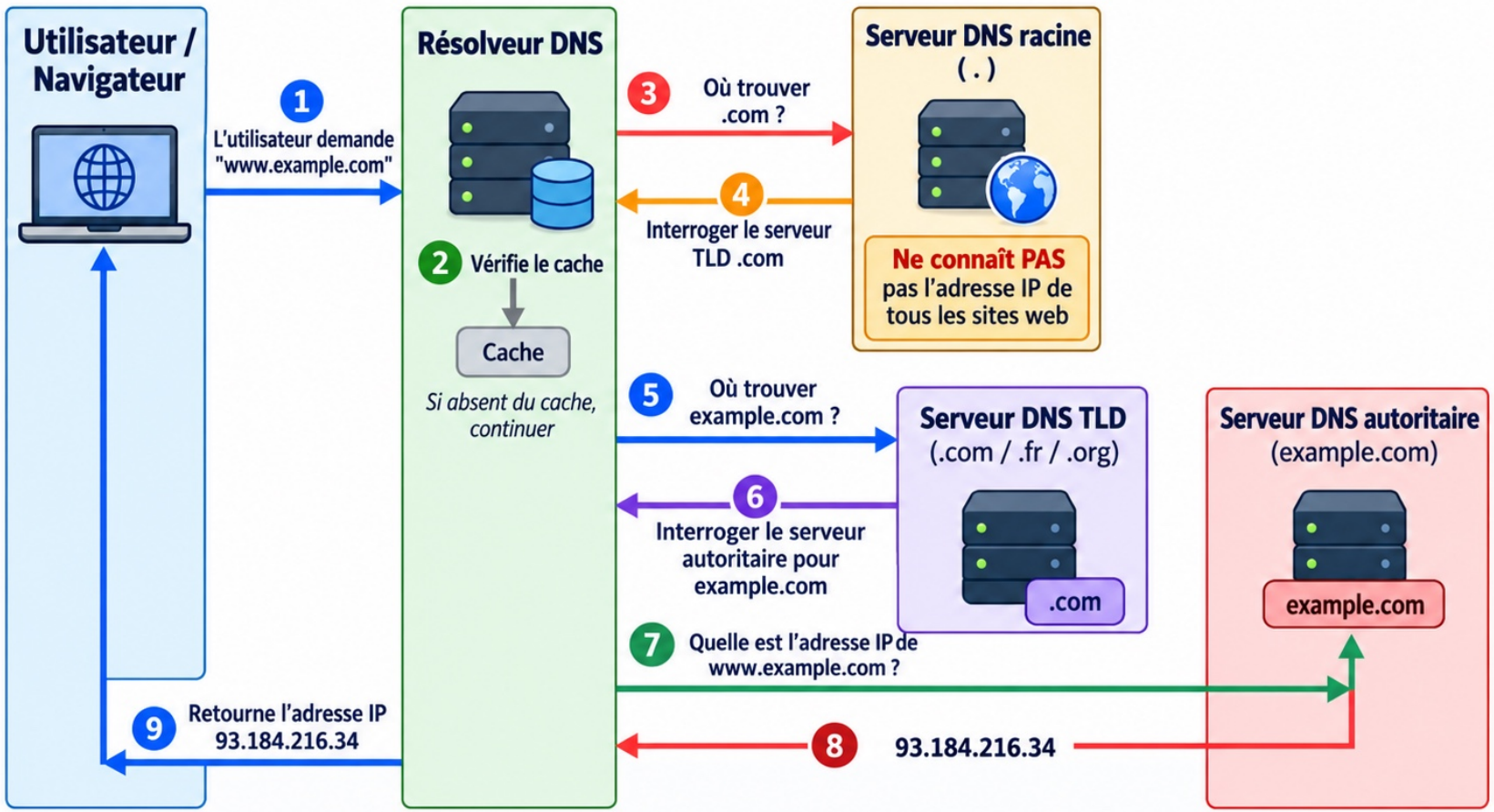
- Prenons un exemple concret : **www.exemple.com.**

www	Nom d'Hôte (ou Sous-domaine)	Identifie un service ou une machine spécifique au sein du domaine (ex: serveur web, serveur de messagerie).
exemple	Nom de Domaine (de deuxième niveau)	Le nom que vous enregistrez (ex: Google, Wikipedia, votre marque).
com	TLD (Top Level Domain ou Domaine de Premier Niveau)	La dernière partie, comme .com, .fr, .org, etc.
Le point final **.**	Racine du DNS (Root)	Représente le niveau le plus élevé de la hiérarchie. Il est souvent omis dans les navigateurs, mais est techniquement présent dans le DNS.

Le service DNS – L'arborescence



Processus de résolution DNS



Idée clé : Le serveur racine démarre la recherche en orientant le résolveur vers le bon serveur de domaine de premier niveau ; il ne stocke pas toutes les adresses IP des sites web.

Map of the Root Servers



• **Racine:** C'est le point de départ de toute recherche DNS. C'est comme le répertoire téléphonique mondial.

- **13 serveurs DNS racines**

- Pour garantir la vitesse, la redondance et la résilience de l'Internet mondial, chaque identité (A à M) est répliquée sur plusieurs centaines, voire des milliers, de serveurs physiques répartis dans le monde entier.

Liste des 13 serveurs racines DNS et leurs opérateurs

Lettre	Nom de domaine	Organisation opératrice
A	a.root-servers.net	VeriSign, Inc.
B	b.root-servers.net	USC-ISI
C	c.root-servers.net	Cogent Communications
D	d.root-servers.net	University of Maryland
E	e.root-servers.net	NASA Ames Research Center
F	f.root-servers.net	Internet Systems Consortium, Inc.
G	g.root-servers.net	U.S. DOD NIC
H	h.root-servers.net	U.S. Army Research Lab
I	i.root-servers.net	Netnod
J	j.root-servers.net	VeriSign, Inc.
K	k.root-servers.net	RIPE NCC
L	l.root-servers.net	ICANN
M	m.root-servers.net	WIDE Project

Le service DNS – Les enregistrements (RRs)

- La base de données des serveurs de noms est constituée **d'enregistrement de ressources** (**Ressource Records** (RRs)).

- À chaque nom de domaine est donc associé un RR. Il existe plusieurs types de RR :

Type d'enregistrement	Fonction
A	= Adress : nom d'hôte -> adresse IPv4 (le plus usuel)
AAAA	= pointe un nom d'hôte vers une adresse IPv6 de 128 bits
PTR	= PoinTeR : correspondance adresse IP -> nom (résolution inverse)
NS	= Name Server : serveur(s) de nom faisant autorité sur le domaine
CNAME	= Canonical NAME : Crée un alias pour un autre nom d'hôte
MX	= Mail eXchange : indique le serveur de messagerie
SOA	= Start Of Authority : contient les informations administratives sur l'annuaire

Le service DNS – Les enregistrements (RRs)

1. A (Address Record) : Associe un nom de domaine à une adresse IPv4.

```
example.com. IN A 93.184.216.34
```

2. AAAA (IPv6 Address Record) : Comme l'enregistrement A, mais pour une adresse IPv6.

```
example.com. IN AAAA 2606:2800:220:1:248:1893:25c8:1946
```

3. PTR (Pointer Record) :Utilisé pour la résolution inverse (IP → nom de domaine).

```
34.216.184.93.in-addr.arpa. IN PTR example.com.
```

4. NS (Name Server Record) : Spécifie les serveurs DNS autoritaires pour un domaine.

```
example.com. IN NS ns1.example.com.
example.com. IN NS ns2.example.com
```

5. CNAME (Canonical Name Record) : Fait d'un domaine un alias d'un autre.

```
www.example.com. IN CNAME example.com.
```

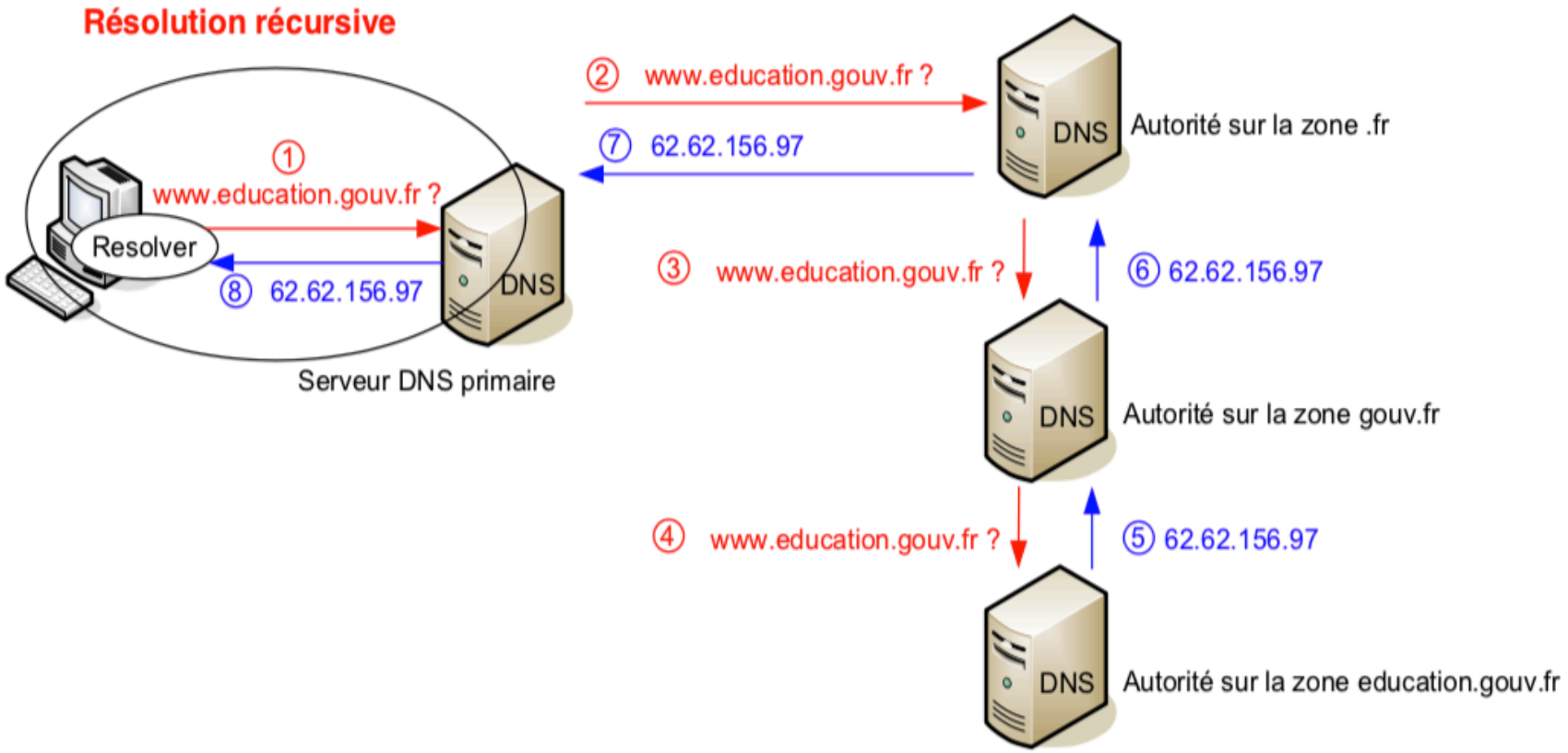
6. MX (Mail Exchange Record) : Indique les serveurs de messagerie pour un domaine.

```
example.com. IN MX 10 mail.example.com.
```

7. SOA : Il indique l'autorité principale pour une zone DNS donnée et contient des informations essentielles sur la gestion de cette zone.

```
example.com. IN SOA ns1.example.com. admin.example.com.
(
    2025050401 ; Serial
    3600       ; Refresh (1 heure)
    600        ; Retry (10 minutes)
    1209600    ; Expire (14 jours)
    86400      ; Minimum TTL (1 jour)
)
```

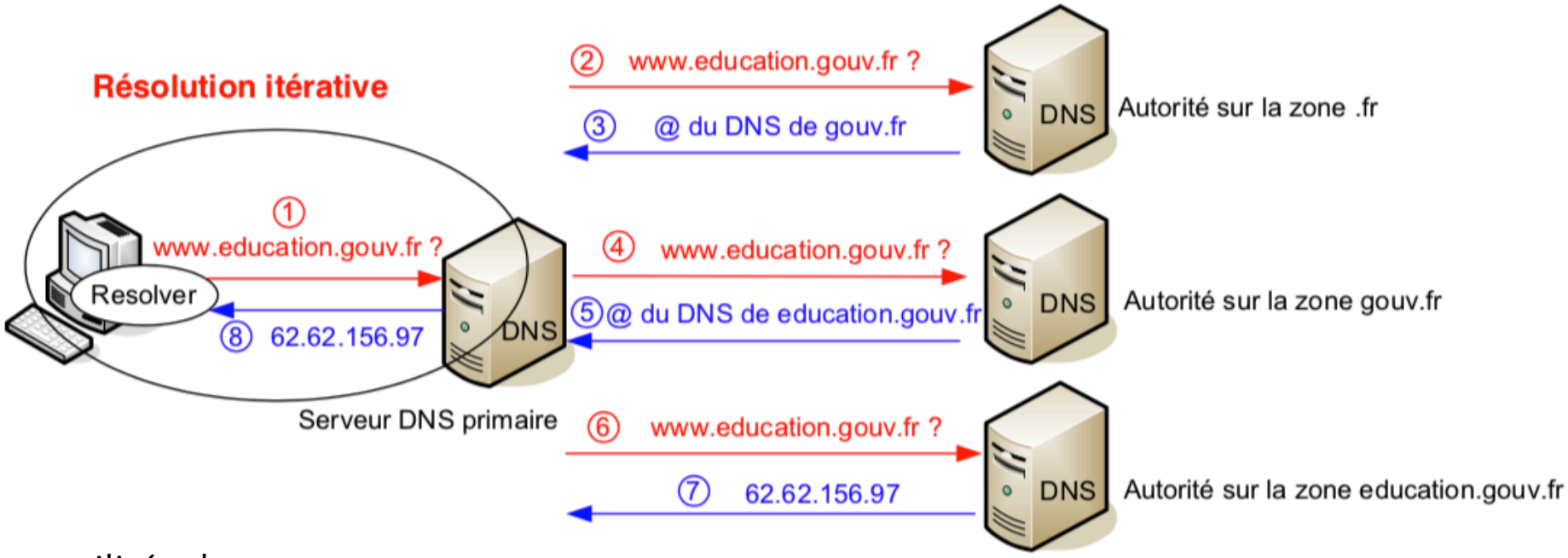
Le service DNS – Résolution récursive



- Si le nom ne peut être résolu localement (zone distante ou nom absent du cache), le serveur primaire transmet à un serveur distant ayant autorité sur le domaine racine concerné (.fr, .com, .net...).
- La requête est ensuite relayée de manière récursive jusqu'à atteindre le serveur DNS ayant autorité sur la zone demandée. L'adresse IP de la machine est alors renvoyée.
- Si le serveur primaire ne connait pas l'adresse ayant autorité sur le domaine de premier niveau, il doit au préalable interroger un serveur racine.

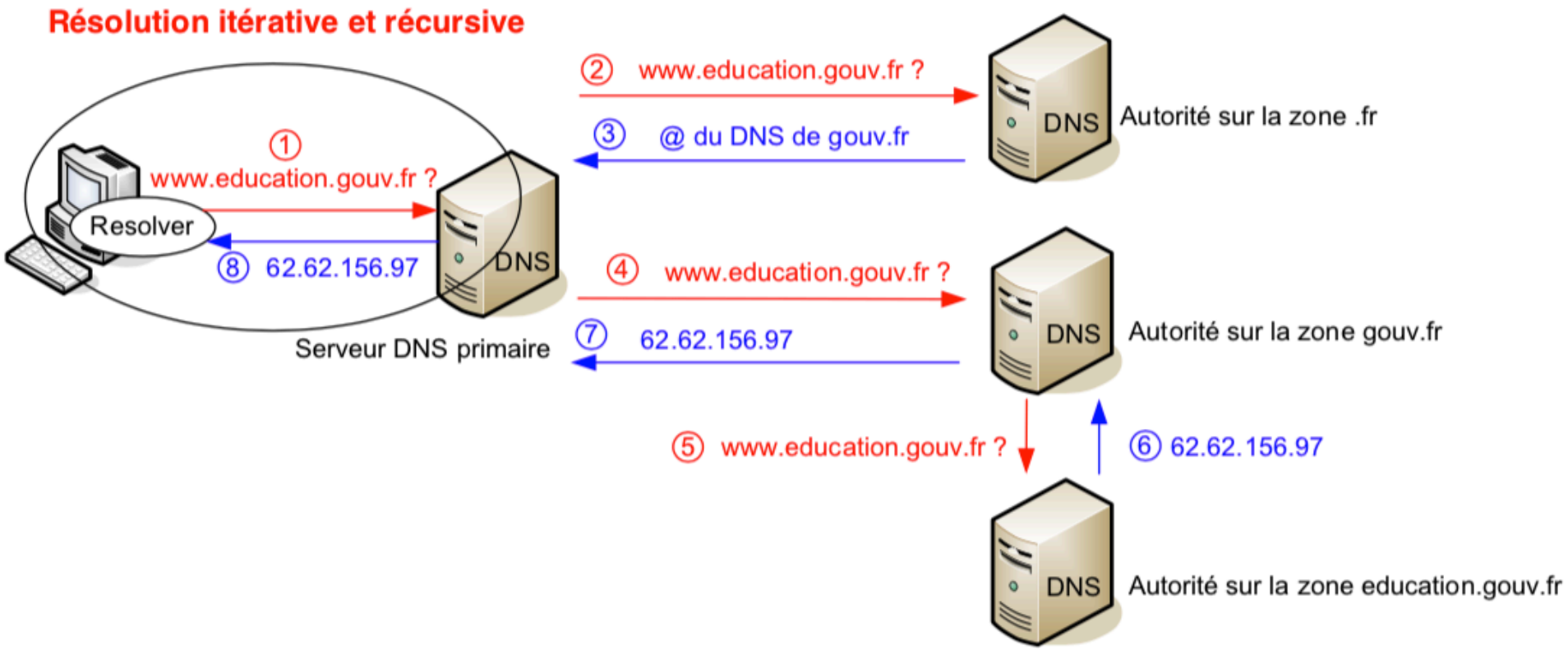
Le service DNS – Résolution itérative

- Le protocole DNS autorise également l'usage de recherches itératives.



- Les recherches itératives sont utilisées lorsque :
  - Le client demande l'utilisation de la récursivité, mais celle-ci est désactivée sur le serveur DNS ;
  - Le client ne demande pas l'utilisation de la récursivité.
- Lorsqu'un serveur DNS n'est pas en mesure de relayer la requête de manière récursive il transmet au serveur primaire l'adresse du prochain serveur.
- Dans l'exemple suivant, toutes les recherches sont de type itératif, c'est le serveur primaire qui est chargé de toutes les requêtes.

Le service DNS – Résolution récursive & itérative



- Dans certains cas, le serveur de nom de premier niveau, fortement sollicité, ne relaye pas les requêtes de manière récursive.
- Seule la requête vers le serveur de nom de premier niveau est itérative, les autres sont récursives.

Protocole : DHCP

Protocole : DHCP

- **DHCP : D*ynamic Host Configuration Protocol***

  Service de configuration dynamique

❖ Qu'est-ce que DHCP :

- Permet de transmettre des paramètres de configuration à des machines d'un réseau TCP/IP.
- Construit sur un **modèle client/serveur.**
- Utilise les ports 67 et 68.

❖ Principe de fonctionnement :

- Le serveur DHCP dispose **d'une plage d'adresses** IP à attribuer.
- Quand un client en demande une adresse IP pour la première fois, le serveur lui en fournit une qu'il n'a pas encore utilisée.
- S'il a déjà distribué toutes ses adresses IP, il réutilise celle d'un client qui n'est plus connecté au réseau.
- C'est un protocole construit comme une extension de **BOOTP** (*Bootstrap Protocol*)

Protocole : DHCP

❖ **Les échanges DHCP :**

- **TCP ou UDP ? Et Pourquoi ?**

Les messages du protocole DHCP utilisent **UDP** au niveau transport.

==> Suffisant pour des échanges simples sur le réseau local.

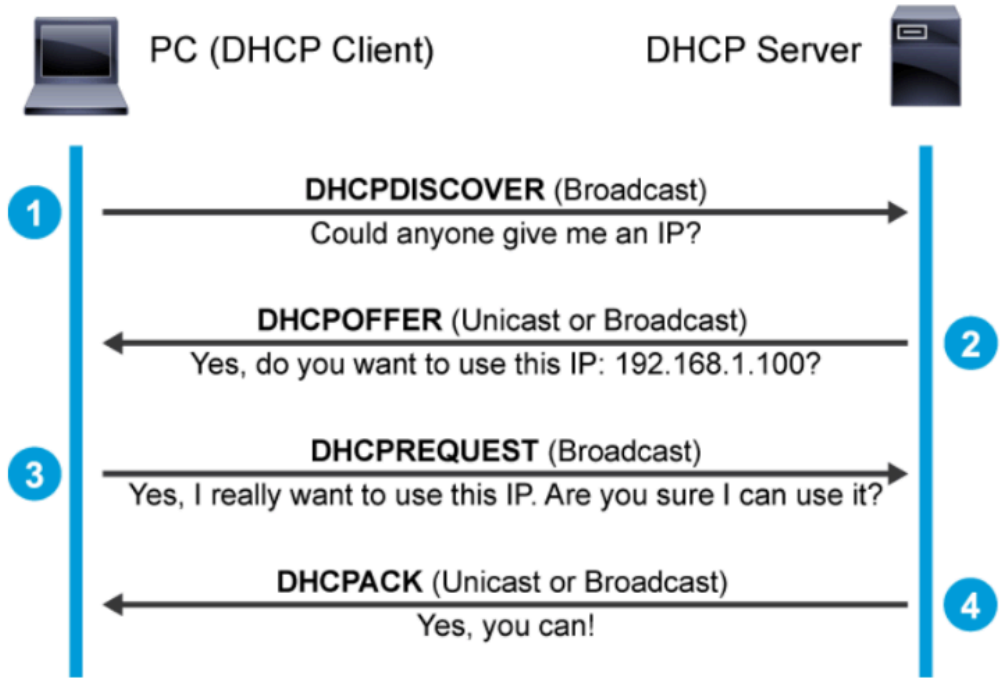
- Le protocole contient plusieurs messages :

1. *DHCPDiscover,*
2. *DHCPOffer,*
3. *DHCPRequest,*
4. *DHCPAck,*
5. *DHCPDecline,*
6. *DHCPRelease.*

La couche application

Protocole : DHCP

1. Une machine cliente qui cherche à obtenir une adresse IP diffuse un ***DHCPDiscover.***
2. Le serveur répond avec un message ***DHCPOffer*** qui contient l'adresse IP proposée au client.
3. Le client diffuse un message ***DHCPRequest*** qui indique l'adresse IP proposée.
4. Le serveur envoie alors la configuration complète au client dans un message ***DHCPAck***.



5. Le client effectue lui aussi un dernier test (avec le protocole ARP). Il cherche, avec une requête ARP, l'adresse MAC de celui qui possède l'adresse IP proposée. Il ne doit obtenir aucune réponse puisque l'adresse IP n'est pas encore distribuée. En cas de problème, il peut refuser la configuration (message ***DHCPDecline***) et recommencer le processus.
6. Enfin, un client quittant le réseau abandonne son adresse par un message ***DHCPRelease***.

Protocole : DHCP

***Bail* ?**

- Pour des raisons d'optimisation des ressources réseau, les serveurs DHCP ne délivrent les adresses IP que pour une durée limitée appelée *bail*.

- Un client dont le bail arrive à terme peut en demander le renouvellement par le message *DHCPRequest*.

ESIEE PARIS

Protocoles :

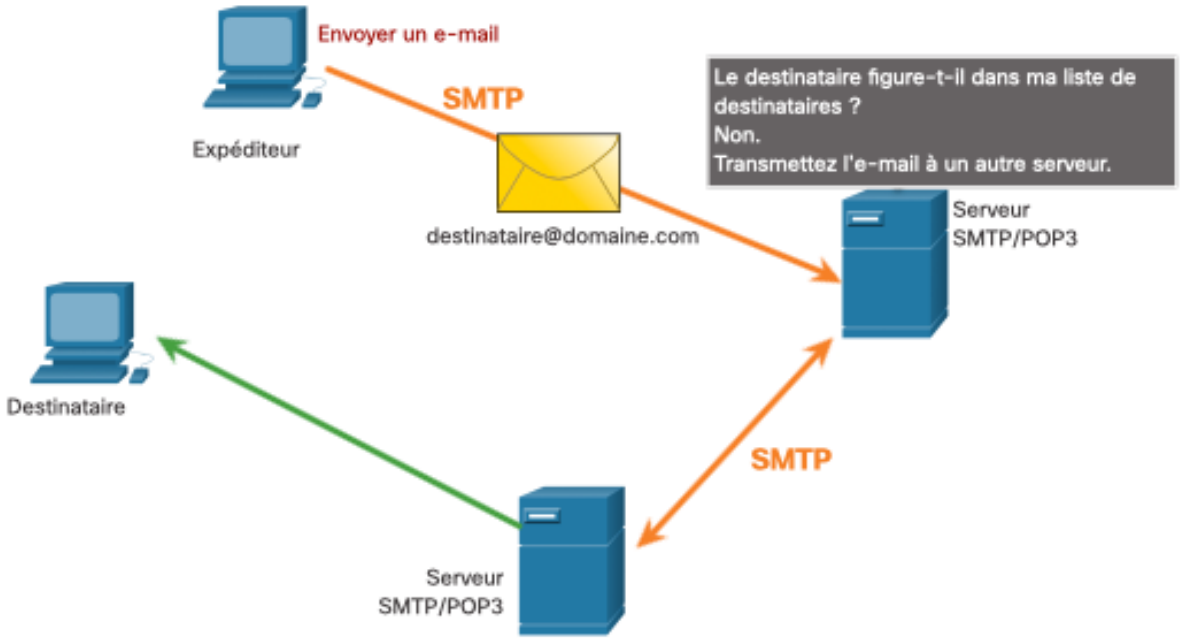
SMTP

POP3/IMAP

La couche application

- **SMTP** (Simple Mail Transfer Protocol)

- **Le port 25** : utilisé par les serveurs de messagerie électronique
- Le **port 587:** par les clients de messagerie.

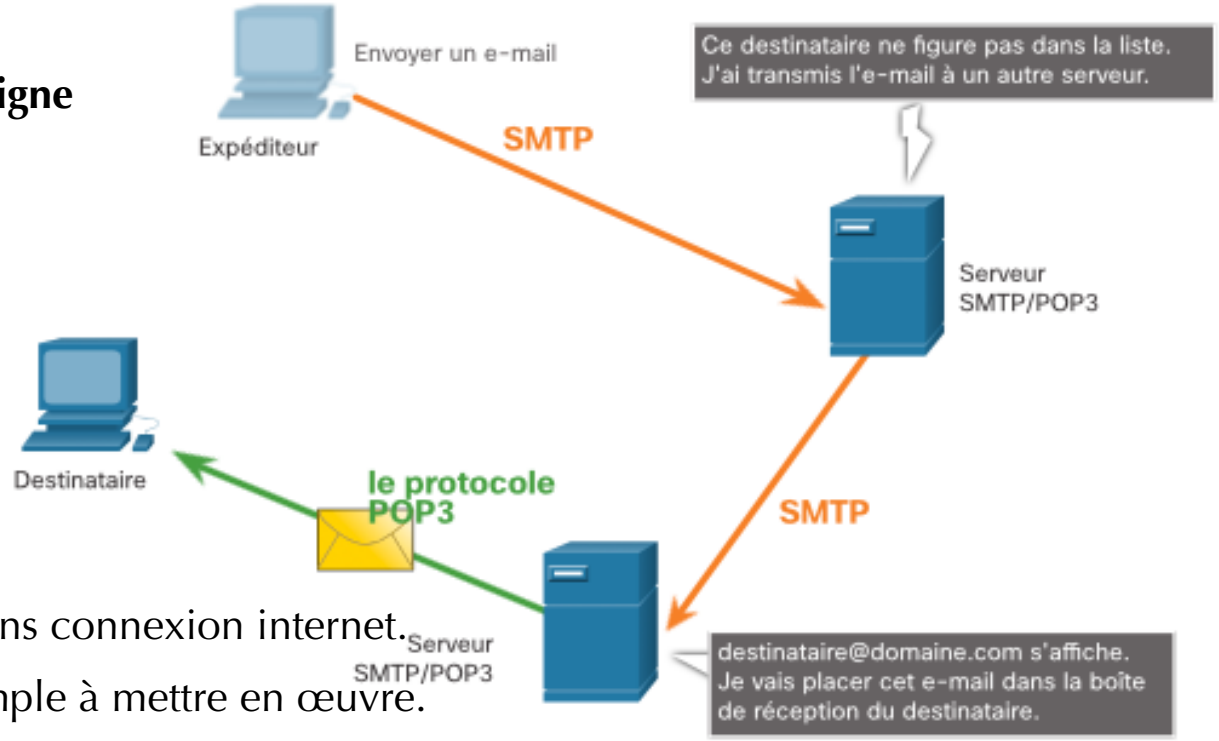


La couche application

- **POP3** (Post Office Protocol)

Le protocole POP3 : télécharger ses emails pour les consulter hors ligne

- **Le fonctionnement :**
  1. **Connexion au serveur**
  2. **Téléchargement des emails:**
  3. **Lecture hors ligne:**



- **Les avantages du POP3 :**
  - **Lecture hors ligne:** Vous pouvez consulter vos emails même sans connexion internet.
  - **Simple d'utilisation:** Le POP3 est un protocole relativement simple à mettre en œuvre.

- **Les inconvénients du POP3 :**
  - **Pas de synchronisation**
  - **Perte de données en cas de problème:**

- **IMAP** (Internet Message Access Protocol).

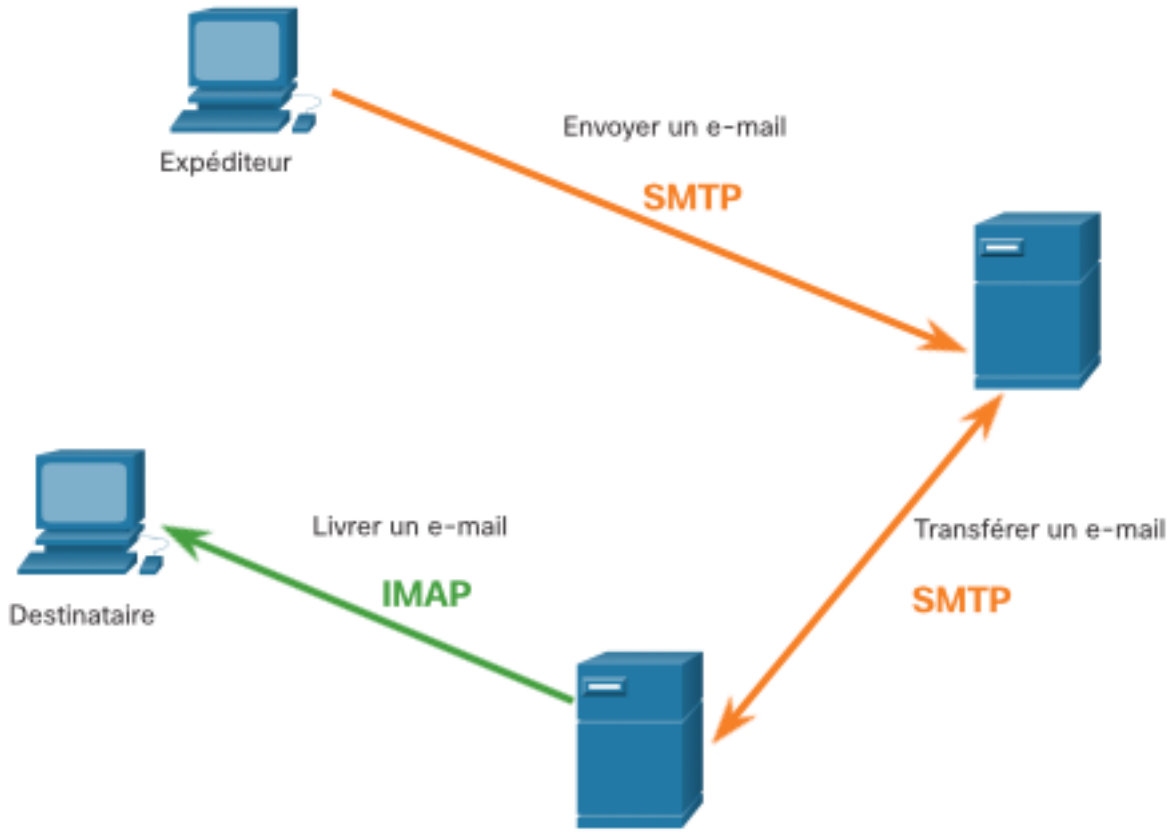
Le protocole IMAP : une boîte mail synchronisée sur tous vos appareils

Les étapes :

1. **Connexion au serveur**
2. **Synchronisation**
3. **Modifications en temps réel**.

Les avantages de l'IMAP :

- **Synchronisation**.
- **Accès à distance**.
- **Plusieurs appareils**.
- **Conservation des emails sur le serveur**



- **Comparaison POP3/IMAP :**

Caractéristique	IMAP	POP3
Synchronisation	Oui	Non
Conservation des emails	Sur le serveur	Sur l'appareil
Accès multiple	Oui	Non

Protocole : FTP

La couche application

Qu'est-ce que le FTP?

- FTP : File,Transfer Protocol
- Un ancien protocole de communication est utilisé pour l'échange de fichiers entre un serveur et un client
- FTP s'appuie sur TCP
- FTP utilise le port 21 par défaut pour la connexion de commande
- Le port 20 pour la connexion de données

Les canaux de communication ?

- **Canal de contrôle:** Pour les commandes (connexion, déconnexion, transfert de fichiers, etc.).
- **Canal de données:** Pour le transfert effectif des fichiers.

Pourquoi deux canaux ?

- **Séparation des rôles :** protocole plus clair et modulaire.
- **Efficacité :** On peut envoyer plusieurs commandes sans attendre la fin d'un transfert.

La couche application

Les limites du FTP ?

- **Sécurité:** Le FTP n'est pas sécurisé par défaut. Les données transitent en clair, ce qui les rend vulnérables aux interceptions.
- **Authentification basique:** Les identifiants (nom d'utilisateur et mot de passe) sont transmis en clair.

Alternatives sécurisées au FTP ?

- **FTPS:** FTP sur SSL/TLS, offrant une connexion chiffrée.
- **SFTP:** Protocole SSH File Transfer Protocol, offrant une sécurité renforcée et des fonctionnalités supplémentaires

ESIEE PARIS

Protocoles :

Telnet

SSH

Telnet

- Telnet : Telecommunication Network
- L'un des premiers protocoles réseau utilisés pour permettre la communication entre ordinateurs à distance via une ligne de commande.
- Développé dans les années 1960 pour établir des sessions interactives entre deux systèmes.
- Telnet fonctionne en **mode client-serveur. Le numéro de Port : 23** par défaut.
- **Pas de chiffrement** : Telnet transmet toutes les informations, y compris les mots de passe et les données sensibles, en **texte clair** sur le réseau, ce qui représente un gros problème de sécurité.
- Telnet est un protocole simple mais n'est pas sécurisé => remplacé par des alternatives plus sûres comme SSH.

SSH

- SSH : Secure Shell
- SSH est un protocole réseau plus récent, créé dans les années 1990, destiné à remplacer Telnet et à offrir des connexions sécurisées pour la gestion à distance des systèmes informatiques.
- SSH est aussi un protocole **client-serveur.** Port : 22 par défaut.
- Une **authentification sécurisée** (par **mot de passe** ou **clé publique**) est effectuée avant que la session ne commence.
- **Chiffrement** : SSH utilise des algorithmes de chiffrement tels que **RSA, AES, ou ECC** pour protéger les données.
- SSH est un protocole **sécurisé** et **polyvalent** (peut être utilisé non seulement pour la gestion à distance mais aussi pour transférer des fichiers de manière sécurisée)

ESIEE PARIS

Comparaison Telnet vs SSH

Caractéristique	Telnet	SSH
Port	23	22
Chiffrement	NON	OUI
Sécurité	Aucune (texte en clair)	Forte (Chiffrement)
Authentification	Simple (nom d'utilisateur/mot de passe)	Mot de passe ou clé publique/privée
Fonctionnalités	Basique (commande à distance)	Plus riche (tunnels, transfert de fichiers, etc.)
Utilisation actuelle	Déconseillé	Fortement recommandé